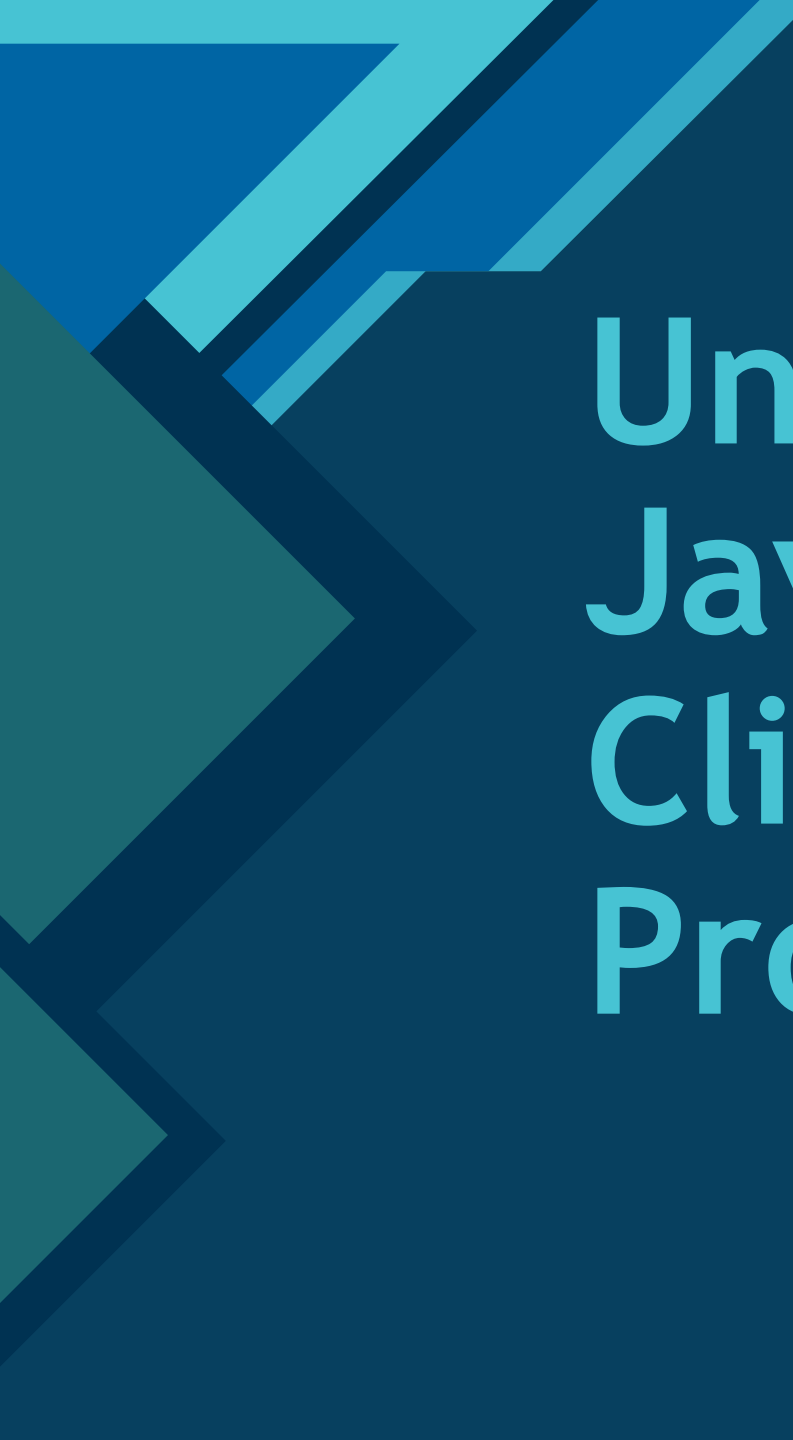




Unit 2: JavaScript and Client-Side Programming

Unit 2: JavaScript and Client-Side Programming (12 hours)

2.1 JavaScript essentials

2.1.1 Data types, variables, control structures, functions

2.1.2 Dom manipulation, events, and form validation

2.1.3 Local storage and session storage

2.1.4 GUI interactions

2.1.5 JavaScript library: jQuery

2.2 Modern JavaScript (ES6+)

2.2.1 Arrow functions, destructuring, spread/rest operators

2.2.2 Callbacks, promises, async/await

2.2.3 Modules and imports

2.3 Client-side applications

2.3.1 Client-side web application development using React JS library

2.3.2 Traditional multi-page apps vs. single-page apps (SPAs)

2.3.3 Component-based UI and props

2.3.4 State management and data flow

Scripting Languages

- A scripting language is a simple programming language designed to enable computer users to write useful programs easily.
- Scripting languages are interpreted, i.e., they are not compiled to any particular machine or operating system. This is what makes them platform-independent.
- Scripts can be written for execution on client or the server. Scripts that get executed on the client machine by a web browser are called client-side scripts whereas those that get executed on the server are called server-side scripts.
- Actually, client-side scripting and server-side scripting languages are the scripting languages generally used to give dynamic effect to the static HTML pages. Static web pages can include forms but for the validation of form's information, we use scripting languages.
- Client side scripting is a process in which scripts are executed by browsers without connecting the server.
- Examples of client-side scripting languages are JavaScript, jQuery, VBScript, Jscript and examples of server-side scripting languages are ASP, PHP, Perl, etc.

Application of Client side Scripting

- Client side scripting is used to make web pages or website more interactive. It is primarily used at the frontend, where the user can interact using the browser.
- Some important applications of client side scripting are listed below:
 - To retrieve data from web browser or user's screen.
 - Used in the field of online games.
 - To customize the web page without reloading the page.
 - Client side scripting is used for validation purpose. If the user enters incorrect credentials on the login page, the web page displays an error message on the client machine without submitting it to the web server.
 - To create ad banners that interact with the user, rather than simply displaying graphics.
 - To create animated images that change when we move the mouse over them.
 - Client side script can be used to detect installed plug-ins and notify the user if a plugin is required.

Advantage of Client side Scripting

- The client side scripting language is quite easy to learn and use. It requires minimum programming knowledge and experience.
- The main advantage of client side scripting is that it is lightweight and relatively easy to implement (syntax not too complex). Editing and executing of the code is fast.
- Data processing is done on the client side from the server, which makes it easier to scale applications with large numbers of users. Thereby, load on the server reduces.
- The client side data validation can be possible using the client side scripting language like JavaScript.
- The execution of client side script is more fast because once the script is downloaded from the server, it is executed by the browser directly on the user's computer.
- Mathematical assessment is also possible using client side scripting.
- The client side programming helps to perform the complex tasks in relatively few steps.
- Script code only executed by the browser without connecting to the server.
- Takes less time to execute script code.
- Browser respond immediately when user presses any key, mouse movement, clicks, etc.

Disadvantage of Client side Scripting

- The main disadvantage of client side scripting is that it is unsecure because the code is sent as is to the client and, therefore, visible to it if the client looks at the sources of his web page. In short, code is usually visible.
- Client side programming cannot be used if we need to access databases or needs to transmit sensitive data over the internet.
- There is no guarantee that user has enabled JavaScript on his computer's browser. Therefore, any required functionality must be loaded on the server despite the possibility that it could be offloaded.
- The smooth running of the script (or program) depends entirely on the client's browser, its configuration, and security level.
- The web application based on the heavy JavaScript can be complicated to debug and maintain.
- Client side scripting languages are usually more limited in functionality than server side scripting languages.

Introduction to JavaScript

- HTML to define the content of web pages; CSS to specify the layout of web pages; JavaScript to program the behavior of web pages.
- JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex animations, clickable buttons, popup menus, etc.).
- There are also more advanced server side versions of JavaScript such as Node.js, which allow you to add more functionality to a website than downloading files (such as real-time collaboration between multiple computers).
- JavaScript contains a standard library of objects, such as Array, Date, and Math, and a core set of language elements such as operators, control structures, and statements.
- JavaScript is a case sensitive scripting language.
- A scripting language is a lightweight programming language.

Introduction to JavaScript (cont....)

- Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects.
- for example:
 - a. Client-side JavaScript: It extends the core language by supplying objects to control a browser and its Document Object Model (DOM).
 - For example, client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation.
 - b. Server-side JavaScript: It extends the core language by supplying objects relevant to running JavaScript on a server.
 - For example, server-side extensions allow an application to communicate with a database, provide continuity of information from one invocation to another of the application, or perform file manipulations on a server.

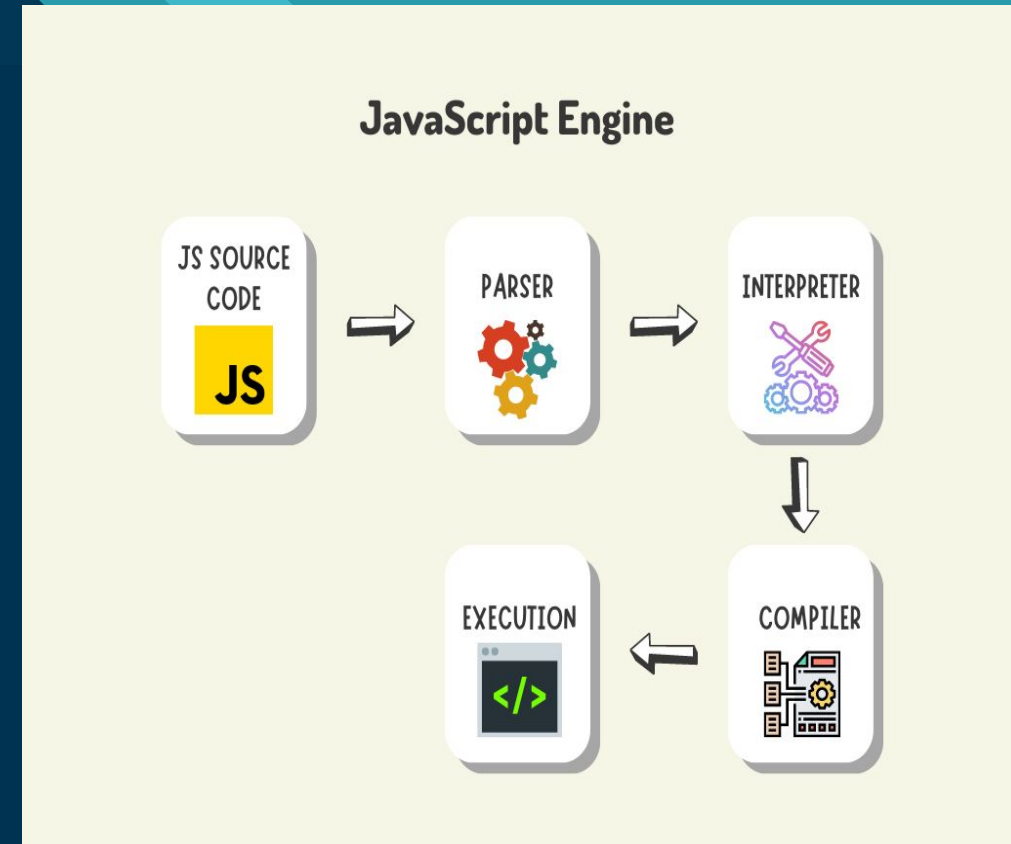
This means that in the browser, JavaScript can change the way the webpage (DOM) looks. And, likewise, Node.js JavaScript on the server can respond to custom requests sent by code executed in the browser.

Introduction to JavaScript (cont....)

- JavaScript is usually embedded directly into HTML pages.
- JavaScript is an interpreted language (that scripts execute without preliminary compilation).
- Everyone can use JavaScript without purchasing a license.
- Java and JavaScript are two completely different languages in both concept & design.
- Java (developed by Sun Microsystems) is a powerful and much more complex programming language – in the same category as C and C++.
- JavaScript was created by Brendan Eich in 1995 during his time at Netscape Communications. It was inspired by Java, Scheme and Self. Netscape, for a time, made the best browser in the world and enjoyed market dominance.

Introduction to JavaScript (cont....)

- JavaScript programming language runs within a web browser.
- The browser's specific engine, known as the JavaScript engine is responsible for rendering the JavaScript code for executing specific instructions.
- The main role behind the successful operation of Browser JavaScript is of JavaScript engine.
- JavaScript engines conduct the following processes to execute JavaScript code in the browser to attain interactive and user-friendly web pages.
- **Parsing:** If any web page contains the JavaScript code, once it is loaded, the parser reads and converts the JavaScript code into a structure called Abstract Syntax Tree (AST), which is also referred to as the code's syntax.
- **Compiling:** Once the JavaScript code has been converted into an AST, it is then either compiled into bytecode or machine code, depending on the target platform. This is done because bytecode is faster to execute and machine code is the native language that the machine hardware understands.
- **Executing:** After the JavaScript code is compiled it is all set to be executed in the browser.
- **Interacting with Browser:** JavaScript performs several events in the browser, such as manipulating API, handling DOM, making HTTP requests, and more. All these interactions of JavaScript with the browser with the help of APIs and interfaces.



Need for JavaScript

- HTML is a specialized language for formatting pages and creating hyperlinks. It has no resources for validating contents and has no structural commands to allow repetitive execution of parts of a program.
- If we create a form to receive data from user, the HTML simply shows the fields and accepts entry. We cannot check for invalid entries and take action. Such a page or form is called static form.
- There is a need to evaluate the contents of all the field types and return an error or warning i.e. to make a page dynamic. For this task, several scripting languages are used of which JavaScript is widely used.
- Hence, to modify the page according to the will of the user, JavaScript is used as it is also supported by most browsers.

What can JavaScript do?

- **JavaScript gives HTML designers a programming tool** – HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax. Almost anyone can put small “snippets” of code into their HTML pages.
- **JavaScript can put dynamic text into an HTML page** – JavaScript statement like this: `document.write("<h1>"+name+"</h1>")` can write a variable text into an HTML page.
- **JavaScript can react to events** – JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element.
- **JavaScript can read and write HTML elements** – JavaScript can read and change the content of an HTML element.
- **JavaScript can be used to validate data** – JavaScript can be used to validate form data before it is submitted to a server. This saves the server from extra processing.
- **JavaScript can be used to detect the visitor's browser** – JavaScript can be used to detect the visitor's browser, and – depending on the browser – load another page specifically designed for that browser.
- **JavaScript can be used to create cookies** – a JavaScript can be used to store and retrieve information on the visitor's computer.

Working with JavaScript

- There are two ways to include JavaScript in your web program:
 1. Embedding the JavaScript code wholly within the document.
 - JavaScript statements can be included in HTML documents by enclosing the statements between an opening `<script>` tag and a closing `</script>` tag.

```
<script language="JavaScript">  
JavaScript Statements  
</script>
```
 - The script tag may be placed in either the head or the body of an HTML document.
 - Example:

```
<!--example1.html-->  
<html>  
<head>  
<title>JavaScript:Example 1</title>  
</head>  
<body>  
<script language="JavaScript">  
document.write("Hello Nepal!")  
document.write("My Nepal,My Pride.")  
</script>  
</body>  
</html>
```

Working with JavaScript cont...

2. Creating the external file with the JavaScript code.

- Using an external file is very simple: we have to just add the JavaScript file address to the <SCRIPT> command line by means of the SRC attribute.

```
<html>
<head>
<title> Using JavaScript</title>
<script language="JavaScript" src="code.js">
</head>
<body> ... </body>
</html>
```

- Example:

```
<!--example2.html-->
<html>
<head>
<title>JavaScript:Example 2</title>
</head>
<body>
<p></p>

<script language="JavaScript" src="show.js">
```

JavaScript - Syntax

- Use JavaScript to write text on a web page:

```
<body>  
<script type="text/javascript"> document.write("HelloWorld!");  
</script>  
</body>
```

- Add HTML tags to the JavaScript:

```
<body>  
<script type="text/javascript"> document.write("<h1>HelloWorld!</h1>");  
</script>  
</body>
```

JavaScript Comments

- Single line comments:

```
<body>
<h1 id="myH"></h1>
<p id="myP"></p>
<script>
// Change heading:
document.getElementById("myH").innerHTML = "JavaScript Comments";
// Change paragraph:
document.getElementById("myP").innerHTML = "My first paragraph.";
</script>
</body>
```

- Multiple lines comments:

```
<body>
<h1 id="myH"></h1>
```


JavaScript Comments cont...

- Comments are not executed by JavaScript.
- Comments can be added to explain the JavaScript, or to make the code more readable.
- Single line comments start with `//`.
- Multi line comments start with `/*` and end with `*/`.

NoScript Tag

- The `<noscript>` tag defines an alternate content to be displayed to users that have disabled scripts in their browser or have a browser that doesn't support script.
- The `<noscript>` element can be used in both `<head>` and `<body>`. When used inside `<head>`, the `<noscript>` element could only contain `<link>`, `<style>`, and `<meta>` elements.
- example

```
<script>
document.write("Hello World!")
</script>
<noscript>Your browser does not support JavaScript!</noscript>
```

What can JavaScript do? - Example 1

```
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change HTML content.</p>
<button type="button"
onclick='document.getElementById("demo").innerHTML =
"Hello JavaScript!">Click Me!</button>
</body>
</html>
```

Output:

What Can JavaScript Do?

JavaScript can change HTML content.

Click Me!

What Can JavaScript Do?

Hello JavaScript!

Click Me!

What can JavaScript do? - Example 2

```
<!DOCTYPE html>
<html>
<body>
<h2>What Can JavaScript Do?</h2>
<p>JavaScript can change HTML attribute values.</p>
<p>In this case JavaScript changes the value of the src (source) attribute of an
image.</p>
<button
onclick="document.querySelector('.myImage').src='https://s3-us-west-2.amazonaws.
com/s.cdpn.io/93927/pic_bulbon.gif'">Turn on the light</button>

<button
onclick="document.getElementById('myImage1').src='https://s3-us-west-2.amazona
ws.com/s.cdpn.io/93927/pic_bulboff.gif'">Turn off the light</button>
</body>
</html>
```

JavaScript Statements

- JavaScript statements are commands to the browser.

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = "Hello.";
```

```
</script>
```

- JavaScript code is a sequence of statements.

```
<p id="demo"></p>
```

```
<script>
```

```
var x, y, z; // Statement 1
```

```
x = 5; // Statement 2
```

```
y = 6; // Statement 3
```

```
z = x + y; // Statement 4
```

```
document.getElementById("demo").innerHTML = "The value of z is " + z + ".";
```

```
</script>
```

- JavaScript statements are separated with semicolon.

```
<p id="demo1"></p>
```

JavaScript Statements cont...

- Multiple statement on one line is allowed.

```
<p id="demo1"></p>
```

```
<script>
```

```
var a, b,c; a = 5; b = 6;c = a + b;
```

```
document.getElementById("demo1").innerHTML = c;
```

```
</script>
```

- JavaScript statements can be grouped together in code blocks.

```
<button type="button" onclick="myFunction()">Click Me!</button>
```

```
<p id="demo1"></p>
```

```
<p id="demo2"></p>
```

```
<script>
```

```
function myFunction()
```

```
{ document.getElementById("demo1").innerHTML = "Hello Dolly!";
```

```
document.getElementById("demo2").innerHTML = "How are you?"
```

JavaScript Types and Variables

- Unlike Java and some other programming languages, JavaScript does not require you to specify the type (int, float, char etc.) of data contained in a variable.
- In fact, the same variable may be used to contain a variety of different values, such as the text string, integer, floating-point value or logical value.
- JavaScript frees you from having to explicitly specify the type of data contained in variable and from having to convert from one data type to another automatically. JavaScript supports following data types:
 - Number - Consists of integer and floating-point number.
 - Boolean - Consists of logical values true and false.
 - String - Consists of string values.
 - Null - Consists of a single value, null.

JavaScript Variables

- JavaScript variables are "containers" for storing information.
- To declare a variable in JavaScript, use the var (optional) keyword:
e.g., `var carname;`
- After the declaration, the variable is empty (it has no value).
- To assign a value to the variable, use the equal sign:
e.g., `carname="Volvo";`
- We can declare many variables in one statement as:
e.g., `var y="Doe", age=30, job="carpenter";`
- Variables can have short names (like x and y) or more descriptive names (age, sum, totalvolume). Some rules are:
 - Variable names must begin with a letter
 - Variable names can also begin with \$ and _ (but not used often)
 - Variable names are case-sensitive (y and Y are different variables)
- Note: "semicolon" is optional in JavaScript.

JavaScript Variables Example

```
<!--example5.html-->
<html>
<head>
<title>JavaScript:Example 5</title>
</head>
<script language="JavaScript">
  var $motto = "<H1>My Nepal,My Pride.</H1>";
  var _text = "<P>Welcome to <B><U><I>Himalayan Country</I></U></B>.</P>";
  abc = "JustTesting";
</script>
<body>
<script language="JavaScript">
  document.write($motto)
  document.write(_text)
  document.write(abc)
</script>
</body>
</html>
```

JavaScript Variables cont...

- **Local JavaScript Variables:**

- A variable declared within a JavaScript function becomes LOCAL and can only be accessed within that function (the variable has local scope).
- You can have local variables with the same name in different functions, because local variables are only recognized by the function in which they are declared. Local variables are destroyed when you exit the function.

- **Global JavaScript Variables:**

- Variables declared outside a function becomes GLOBAL, and all scripts and functions on the web page can access it.
- Global variables are destroyed when you close the page. If you declare a variable, without using "var", the variable always becomes GLOBAL.

- Example:

```
function myfunction(){  
    msg = "Hello JavaScript!";  
}
```

```
myfunction();  
alert(msg); // msg becomes global variable so can be accessed here
```

JavaScript Variables cont...

- **Assigning Values to Undeclared JavaScript Variables:**

- If you assign values to variables that have not yet been declared, the variables will automatically be declared as global variables.

For example, `x=5;`

`carname="Volvo";`

Here the variables `x` and `carname` are global variables (if they don't already exist).

Keywords in JavaScript

- Var keyword:
 - Every programming language have a method to declare variable to store anything else, to process that. So, in the JavaScript, we used a var keyword to create a variable and then store data into that variable.
 - Example:

```
var b = 30;  
var c =23;  
document.write(b+c);
```
- Const keyword:
 - It is used to declare and initialize constant.
 - We cannot assign a value to constant again.

```
const x = 4.3  
x = 2.3;    // It will give an error  
x = x + 13; // This will also give an error
```
 - We must declare and assign the value at time of declaration.

```
const y;  
y = 20; //error
```

Keywords in JavaScript cont...

- Let Keyword:

- It declare a variable with limited scope.
- Variable will be used only that block in which it is declared.
- For example, in specific block of code as in if else structure, function, loop etc.

```
function Test() {  
    let y = 12;  
    document.write(y); //give 12  
}  
Test();  
document.write(y); // give error of not defined
```

The Lifetime of JavaScript Variables

- If you declare a variable, using "var", within a function, the variable can only be accessed within that function. When you exit the function, the variable is destroyed. These variables are called local variables. You can have local variables with the same name in different functions, because each is recognized only by the function in which it is declared.
- If you declare a variable outside a function, all the functions on your page can access it. The lifetime of these variables starts when they are declared and ends when the page is closed .

JavaScript Examples

```
<!--example6.html-->
<html>
<head>
<title>JavaScript:Example 6</title>
</head>
<script language="JavaScript">
  document.write("12345679 * 72 = ")
  // Multiplication
  document.write(12345679*72)
  document.write("<BR>")
  document.write("12e2 = ")
  // Positive Exponential
  document.write(12e2)
  document.write("<BR>")
  document.write("12e-2 =")
  // Negative Exponential
  document.write(12e-2)
```

JavaScript Examples cont...

```
<!--example7.html-->
<html>
<head>
<title>JavaScript: Example 7</title>
</head>
<body>
<script language="JavaScript">
  s1 = "Test"
  i1 = 123
  f1 = 123.45
  document.write("<H2>Implicit Conversion</H2>")
  document.write("Adding String(s1) and Integer(i1) : ")
  document.write(s1+i1)
  document.write("<BR>")
  document.write("Adding Integer(i1) and Float(f1) : ")
  document.write(i1+f1)
  document.write("<BR>")
</script>
</body>
</html>
```


JavaScript Examples cont...

```
<!--example8.html-->
```

```
<html>
```

```
<head>
```

```
<title>JavaScript: Example 8</title>
```

```
</head>
```

```
<body>
```

```
<script language="JavaScript">
```

```
document.write("<H2>Explicit Conversion</H2>")
```

```
document.write("<B><U>Convert String to Numeric</U> :</B> ")
```

//The eval() function evaluates JavaScript code represented as a string and returns its completion value. The source is parsed as a script.

```
document.write('eval("12.34*10") = ')
```

```
document.write(eval("12.34*10"))
```

```
document.write("<BR>")
```

```
document.write("<B><U>Convert String to Integer</U> :</B> ")
```

//The parseInt method parses a value as a string and returns the first integer.

```
document.write('parseInt("0xff") = ')
```

```
document.write(parseInt("0xff"))
```

JavaScript Reserved Words

- Every Programming language have a list of special words. So that special words have special meanings. Like if, else, do, while, for etc. You cannot use that words as variable name, function, class name etc.

abstract	else	instanceof	switch
boolean	enum	int	synchronized
break	export	interface	this
byte	extends	long	throw
case	false	native	throws
catch	final	new	transient
char	finally	null	true
class	float	package	try
const	for	private	typeof
continue	function	protected	var
debugger	goto	public	void
default	if	return	volatile
delete	implements	short	while
do	import	static	with
double	in	super	

JavaScript Output / Input

- Output Function

- To display anything else on the browser, we used following function
document.write()
console.log() //This method is used to see error message in the console section

- Input Function

- To get anything else from the user, we use following method
window.prompt()

JavaScript Operators

- Unary Operators:

- ++ Increment current value by 1
- -- Decrement current value by 1

- Types of Unary Operators:

- a++ Post Increment
- ++a Pre Increment
- a-- Post Decrement
- --a Pre Decrement

- Binary Operators:

- + Addition
- - Subtraction
- / Division
- * Multiplication
- % Modulus

JavaScript Operators cont...

- Logical Operators:
 - && Logical AND
 - || Logical OR
 - ! Logical NOT
- Relational Operators:
 - == Equal
 - === Strictly Equal
 - != Not Equal
 - !== Strictly Not Equal
 - < Less Than
 - <= Less Than or Equal
 - > Greater Than
 - >= Greater Than or Equal
 - Example: The equal (==) and not equal (!=) operators perform type conversions before testing for equality. For instance, "5" == 5 evaluates to 'true'. The strictly equal (===) and strictly not equal (!==) operators do not perform type conversions before testing for equality. For instance, "5" === 5 evaluates to 'false' and "5" !== 5 evaluates 'true'.

JavaScript Operators cont...

- Assignment Operators:

- = Assign right value to left variable
- += Assign right value to left variable with incrementing right value by left value
- -= Assign right value to left variable with decrementing right value by left value
- *= Assign right value to left variable with multiplying right value by left value
- /= Assign right value to left variable with dividing right value by left value
- %= Assign right value to left variable with taking modulus of right value by left value

- Conditional Operator:

- A conditional operator assigns a value to a variable based on some condition.
- Syntax:
variablename=(condition)?value1:value2
- Example: If the variable age is a value below 18, the value of the variable voteable will be "Too young", otherwise the value of voteable will be "Old enough":
voteable=(age<18)?"Too young":"Old enough";

Conditional Statements

- Two types:
 1. Decision-making statements
 2. Loops

1. Decision-Making Statements:

In JavaScript we have the following conditional statements:

- **if statement** - use this statement to execute some code only if a specified condition is true. if is used to check for a condition whether its true or not. Condition could be any expression that returns true or false. When condition satisfies then statements following if statement are executed.

- Syntax:

```
if(condition) {  
    statement1  
    statement2  
    ... }  
}
```

- Example:

```
<script>  
if(5>4) {  
    document.write("yes 5 is greater than 4");  
    document.write("<br />" + "JavaScript is fun");  
    document.write("<br />"); }  
</script>
```

Decision-Making Statements cont...

- **if...else statement** - use this statement to execute some code if the condition is true and another code if the condition is false.

- Syntax:

```
if(condition)
{
    statements
}
else
{
    statements
}
```

- Example:

```
<script>
if(3>4)
{
    document.write("True");
}
else
{
    document.write("False");
}
//outputs False
</script>
```


Decision-Making Statements cont...

- **else if statement** - use this statement to select one of many blocks of code to be executed. if you want that if one condition satisfies then don't perform further conditional checks. At this point else if statement is what you need.

- Syntax:

```
if(condition)
{
    statements
}
else if(condition)
{
    statements
}
else
{
    statements
}
```

- Example:

```
<script>
var a=3;
var b=4;
if(a > b)
{
```

Decision-Making Statements cont...

- **switch statement** - use this statement to select one of many blocks of code to be executed.

- Syntax:

```
switch(expression)
{
    case value-1: statements; break;
    case value-2: statements; break;
    .....
    case value-n: statements; break;
    default: statements; break;
}
```

- Example:

```
<script>
var a = 3;
switch(a)
{
    case 1: document.write("one"); break;
    case 2: document.write("two"); break;
    case 3: document.write("three"); break;
    case 4: document.write("four"); break;
    case 5: document.write("five"); break;
    default: document.write("number not found");
}
//outputs three
</script>
```

- case value can be any number, character or string. Keep this in mind.

Loops

- Loops are used to execute a specific statement for a given number of times. Loops are always followed by some condition. JavaScript provides all the basic loops that other programming languages have.

- **while loop:**

while loop checks for a given condition to be true to execute statements that it contains within it. When the condition becomes false, while loop break the execution of statements.

- Syntax:

```
initializer; //initializer is a variable that is tested within the condition.  
while(condition)  
{  
    statement-1;  
    statement-2;  
    .....  
    modifier; //modifier modifies the initializer so that the condition becomes false at some point.  
}
```

- Example:

```
<script>  
var i = 0;  
while(i<6)  
{  
    document.write(i);  
    document.write("<br />");  
    i++;  
}  
</script>
```

Loops cont...

- **do while loop:**

do while loop first execute the statements that it contain and then check for a condition. So even if the condition is false do-while loop is going to run at least once.

- Syntax:

```
initializer;
do
{
    statement-1;
    statement-2;
    .....
    modifier;
}
while(condition);
```

- Example:

```
<script>
var i = 5;
do
{
    document.write(i);
    document.write("<br />");
    i++;
}
while(i<6);
</script>
```

Loops cont...

- **for loop:**

for loop is generally used when the number of iterations/repetitions are already known. Its not that such tasks can't be done by while loop but its just a matter of preference.

- Syntax:

```
for(initializer; condition; modifier)  
{  
    statement-1;  
    .....  
}
```

- Example:

```
<script>  
for(var i=0;i<10;i++)  
{  
    document.write(i);  
    document.write("<br />");  
}  
</script>
```

Break and Continue

- **Break:**

The break statement can also be used to jump out of a loop.

- Example:

This example jumps out of the loop when x is equal to 4:

```
<script>
let text = "";
for (i= 0; i< 10; i++) {
  if (i== 4) {
    break;
  }
  text += "The number is " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
```

- **Continue:**

The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

- Example

Popup Boxes

- **Alert Box:**

- An alert box is often used if you want to make sure information comes through to the user.
- When an alert box pops up, the user will have to click “OK” to proceed.

alert("sometext");

- **Example:**

```
<head>
```

```
<script type="text/javascript">
```

```
    function show_alert(){
```

```
        alert("I am an alert box!"); }
```

```
</script>
```

```
</head>
```

```
<body>
```

```
    <input type="button" onclick="show_alert()" value="Show alertbox" />
```

```
</body>
```

Popup Boxes cont...

- **Confirmation Box:**

- A confirm box is often used if you want the user to verify or accept something.
- When a confirm box pops up, the user will have to click either “OK” or “Cancel” to proceed.
- If the user clicks “OK”, the box returns true; and if the user clicks “Cancel”, the box returns false.

```
confirm("sometext");
```

- Example:

```
<head>
<script type="text/javascript">
    function show_confirm(){
        var r=confirm("Press a button");
        if (r==true) {
            document.write("You pressed OK!"); }
        else {
            document.write("You pressed Cancel!"); }
        }
</script>
</head>
<body>
<input type="button" onclick="show_confirm()" value="Show confirm box" />
</body>
```


Popup Boxes cont...

- **Prompt box:**

- A prompt box is often used if you want the user to input a value before entering a page.
- When a prompt box pops up, the user will have to click either “OK” or “Cancel” to proceed after entering an input value.
- if the user clicks “OK”, the box returns the input value; and if the user clicks “Cancel”, the box returns null.

```
prompt("sometext", "defaultvalue");
```

- **Example:**

```
<body>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
var txt;
var person = prompt("Please enter your name:", "Harry Potter");
if (person == null || person == "") {txt = "User cancelled the prompt.";}
else {txt = "Hello " + person + "! How are you today?";}
document.getElementById("demo").innerHTML = txt;
}
</script>
</body>
```

Functions

- A function is simply a block of code with a name, which allows the block of code to be called by other components in the scripts to perform certain tasks.
- Functions can also accept parameters that they use to complete their task.
- JavaScript actually comes with a number of built-in functions to accomplish a variety of tasks.

- Creating Custom Functions:

```
function name_of_function(argument1,argument2,...,arguments)
{
  //Block of code
}
```

- Calling Functions:

- There are two common ways to call a function; from an event handler and from another function.
- Calling a function is simple. You have to specify its name followed by the pair of parenthesis

Functions cont...

- A Simple Function Example:

```
<html>
<head>
<script>
function myFunction()
{
document.getElementById("demo").innerHTML = "HelloWorld!";
}
</script>
</head>
<body>
<p>When you click "Try it", a function will be called.</p>
<p>The function will display a message.</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
</body>
</html>
```

Functions cont...

- Function that Returns a Value Example:

```
<html>
<body>
<p>This example calls a function which performs a calculation and
returns the result:</p>
<p id="demo"></p>
<script>
var x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;
function myFunction(a, b)
{
  return a * b;
}
</script>
</body>
</html>
```

Immediately Invoked Function Expression(IIFE)

- An IIFE (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined.
- The name IIFE is promoted by Ben Alman.
- It is a design pattern which is also known as a Self-Executing Anonymous Function and contains two major parts:
 - The first is the anonymous function with lexical scope enclosed within the Grouping Operator (). This prevents accessing variables within the IIFE idiom as well as polluting the global scope.
 - The second part creates the immediately invoked function expression () through which the JavaScript engine will directly interpret the function.

Immediately Invoked Function Expression(IIFE) (cont..)

- Syntax:

```
(function () {  
    // Function Logic Here.  
})();
```

- Example:

```
<script>  
    // Regular Function.  
    function Greet() {  
        console.log("Welcome to GFG!");  
    };  
    // Execution of Regular Function.  
    Greet();  
  
    // IIFE creation and execution.  
    (function() {  
        console.log("Welcome to GFG!");  
    })();  
</script>
```

Immediately Invoked Function Expression(IIFE) (cont...)

- Use Cases Of IIFE
 - Avoid polluting the global namespace
 - To create closures
 - Avoid conflict of variable names between libraries and programs.
 - IIFE is used to create private and public variables and methods
 - It is used to execute the async and await function
 - It is used in JQuery Library
 - It is used to work with require function

JavaScript Objects

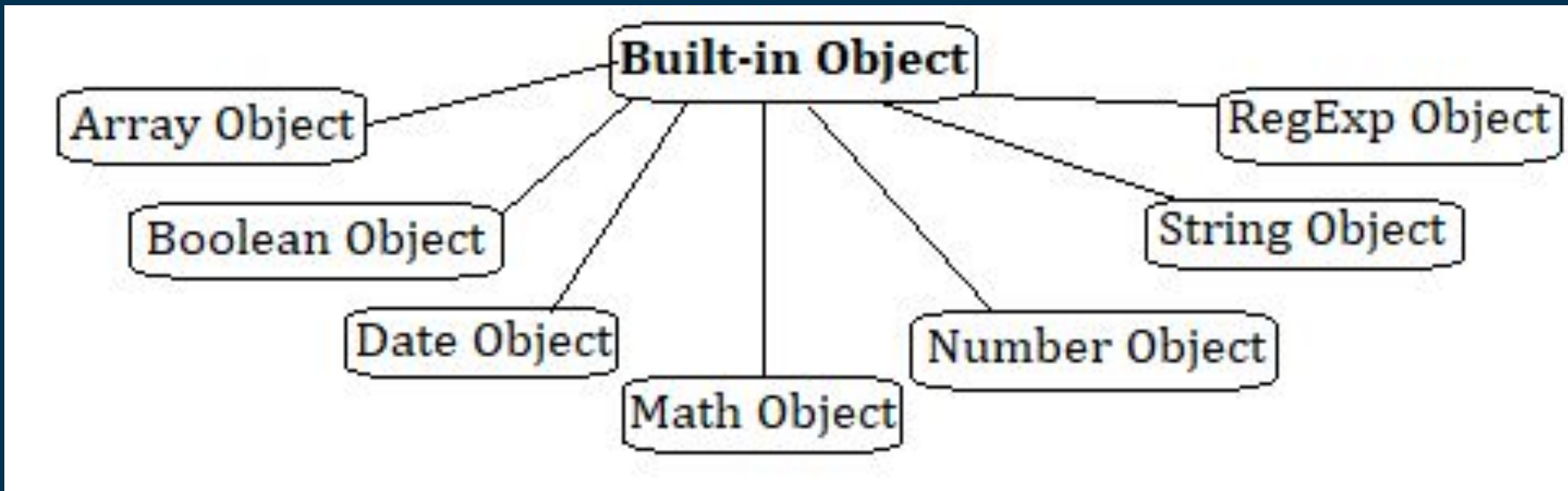
- In JavaScript, almost everything is an object.
- Booleans, Numbers, and Strings (if defined with the new keyword) can be objects.
- Dates, Maths, Arrays, Functions, Regular expressions, and Objects are always objects.
- All JavaScript values, except primitives, are objects.
- A primitive value is a value that has no properties or methods.
- A primitive data type is data that has a primitive value. There are 7 primitive data types: string, number, bigint, boolean, undefined, symbol, and null.
- Primitive values are immutable (they are hardcoded and therefore cannot be changed).

JavaScript Objects Properties

- JavaScript variables can contain single values.
`var person = "John Doe";`
- Objects are variables too; but objects can contain many values.
- The values are written as name:value pairs (name and value separated by a colon).
 - creating an object using object literal `var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};`
- A JavaScript object is a collection of named values.
- The named values, in JavaScript objects, are called properties.
- In above example, "firstName", "lastName", "age", and "eyeColor" are properties; whereas "John", "Doe", "50", and "blue" are their respective values.

JavaScript Built in Objects

- JavaScript has several built-in or core language objects.
- These built-in objects are available regardless of window content and operate independently of whatever page your browser has loaded.



Math Object in JavaScript

- The Math object allows you to perform mathematical tasks.
- The Math object includes several mathematical constants and methods.

```
var pi_value = Math.PI;  
var sqrt_value = Math.sqrt(16);
```

- All properties and methods of Math can be called by using Math as an object without creating it.

```
<body>  
<h2>JavaScript Math.PI</h2>  
<p>Math.PI returns the ratio of a circle's circumference to its diameter:</p>  
<p id="demo"></p>  
<script>  
document.getElementById("demo").innerHTML = Math.PI;  
</script>  
</body>
```

Math Object in JavaScript cont...

Properties

- `Math.E` – returns Euler's number (approx. 2.718)
- `Math.LN2` – returns the natural logarithm of 2 (approx. 0.693)
- `Math.LN10` – returns the natural logarithm of 10 (approx. 2.302)
- `Math.LOG2E` – returns the base-2 logarithm of E (approx. 1.442)
- `Math.LOG10E` – returns the base10 logarithm of E (approx. 0.434)
- `Math.PI` – returns PI (approx. 3.14159)

Methods

- `abs(x)` – returns the absolute value of x
- `ceil(x)` – returns x, rounded upwards to the nearest integer
- `floor(x)` – returns x, rounded downwards to the nearest integer
- `log(x)` – returns the natural logarithm (base E) of x
- `max(x,y,z,...,n)` – returns the number with the highest value
- `min(x,y,z,...,n)` – returns the number with the lowest value
- `pow(x,y)` – returns the value of x to the power of y
- `sqrt(x)` – returns the square root of x
- `random(x)` – returns a random number between 0 & 1
- `round(x)` – rounds x to the nearest integer

Window Object in JavaScript

- The window object represents the browser's window.
- Two properties window.innerHeight and window.innerWidth can be used to determine the size (in pixels) of browser window.

```
<body>
<p id="demo"></p>
<script>
var w = window.innerWidth;
var h = window.innerHeight;
var x = document.getElementById("demo");
x.innerHTML = "Browser inner window width: " + w + ", height: " + h + ".";
</script>
</body>
```

Date Object in JavaScript

- The Date object is used to work with dates and times.
- Date objects are created with the Date() constructor.
- You can use 4 variant of Date constructor to create date object.
 - Date()
 - Date(milliseconds)
 - Date(dateString)
 - Date(year, month, day, hours, minutes, seconds, milliseconds)

- Example:

Current Date and Time:

```
<script>
```

```
var today=new Date();
```

```
document.getElementById('txt').innerHTML=today;
```

```
</script>
```

Date Object in JavaScript

- Date object methods in JavaScript:
 - getDate() – returns the day of the month (from 1 – 31)
 - getDay() – returns the day of the week (from 0 – 6)
 - getFullYear() – returns the year (four digits)
 - getHours() – returns the hour (from 0 – 23)
 - getMilliseconds() – returns the milliseconds (from 0 – 999)
 - getMinutes() – returns the minutes (from 0 – 59)
 - getMonth() – returns the month (from 0 – 11)
 - getSeconds() – returns the seconds (from 0 – 59)

Date Object in JavaScript cont...

- Example:

```
<html>
<body>
<h2>JavaScript new Date()</h2>
<p id="demo"></p>
<script>
var d = new Date();
document.getElementById("demo").innerHTML = d;
//document.getElementById("demo").innerHTML = d.getDate();
//document.getElementById("demo").innerHTML = d.getDay();
//document.getElementById("demo").innerHTML = d.getFullYear();
//document.getElementById("demo").innerHTML = d.getHours();
//document.getElementById("demo").innerHTML = d.getMilliseconds();
//document.getElementById("demo").innerHTML = d.getMinutes();
//document.getElementById("demo").innerHTML = d.getMonths();
//document.getElementById("demo").innerHTML = d.getSeconds();
</script>
</body>
</html>
```


Date Object in JavaScript cont...

- Example to print date/month/year:

```
<html>
<body>
<h2>JavaScript day/month/year</h2>
<script>
var date=new Date();
var day=date.getDate();
var month=date.getMonth()+1;
var year=date.getFullYear();
document.write("<br>Date is: "+day+"/"+month+"/"+year);
</script>
</body>
</html>
```

String Methods in JavaScript

- String methods helps us to work with strings.
- The length property returns the length of a string.
- The indexOf() method returns the index of the first occurrence of a specified text in a string.
- The search() method searches a string for a specified value and returns the position of the match.
- The slice() method extracts a part of a string and returns the extracted part in a new string.
- The replace() method replaces a specified value with another value in a string.
- The toUpperCase() method converts a string to upper case.
- The toLowerCase() method converts a string to lower case.

String Methods in JavaScript cont...

- Example:

```
<html>
<body>
<h2>JavaScript String Properties</h2>
<p id="demo"></p>
<script>
var txt = "The quick brown fox jumps over the lazy dog.";
document.getElementById("demo").innerHTML = txt.length;
/*
var pos = txt.indexOf("fox"); document.getElementById("demo").innerHTML = pos;
var pos = txt.search("jumps"); document.getElementById("demo").innerHTML = pos;
var res = txt.slice(4,9); document.getElementById("demo").innerHTML = res;
var txt = txt.replace("quick","slow"); document.getElementById("demo").innerHTML = txt;
var txt = txt.toUpperCase(); document.getElementById("demo").innerHTML = txt;
*/
</script>
</body>
</html>
```

Number Methods in JavaScript

- The `Number()`, can be used to convert JavaScript variables to numbers.
- The `parseInt()` parses a string and returns a whole number. Spaces are allowed but only the first number is returned.
- The `parseFloat()` parses a string and returns a number. Spaces are allowed but only the first number is returned.
- The `toExponential()` method returns a string representing the number object in exponential notation.
- The `toString()` method returns a string representing the specified object. The `toString()` method parses its first argument, and attempts to return a string representation in the specified radix.
- The `toFixed()` returns a string, with the number written with a specified number of decimals.
- The `toPrecision()` returns a string, with a number written with a specified length.

Number Methods in JavaScript cont...

- Example:

```
<script type="text/javascript">
x = true;
document.write(Number(x) + "<br>");
x = false;
document.write(Number(x) + "<br>");
x = "10"
document.write(Number(x) + "<br>");
x = "10 20"
document.write(Number(x) + "<br>");
document.write(parseInt("12") + "<br>");
document.write(parseInt("12.33") + "<br>");
document.write(parseInt("12 6") + "<br>");
document.write(parseInt("12 years") + "<br>");
document.write(parseInt("month 12"));
</script>
```

Number Methods in JavaScript cont...

- Example:

```
<body>
<p id="demo"></p>
<script>
var x = 9.656;
document.getElementById("demo").innerHTML = x.toExponential() + "<br>" +
x.toExponential(2) + "<br>" + x.toExponential(4) + "<br>" + x.toExponential(6);
</script>
</body>
```

- Example:

```
<script type="text/javascript">
var num = new Number(15);
document.write("num.toString() is:" + num.toString() + "<br />");
document.write("num.toString(2) is " + num.toString(2) + "<br />");
document.write("num.toString(8) is:" + num.toString(8) + "<br />");
</script>
```

Number Methods in JavaScript cont...

- Example:

```
<body>
```

```
<p id="demo"></p>
```

```
<script>
```

```
var x = 9.656;
```

```
document.getElementById("demo").innerHTML = x.toFixed(0) + "<br>" +  
x.toPrecision() + "<br>" + x.toFixed(2) + "<br>" + x.toPrecision(2) + "<br>" +  
x.toFixed(4) + "<br>" + x.toPrecision(4) + "<br>" + x.toFixed(6) + "<br>" +  
x.toPrecision(6);
```

```
</script>
```

```
</body>
```

Boolean Methods in JavaScript

- A JavaScript Boolean represents one of two values: true or false.
- Boolean(5>3) and 5>3 returns same output as true.
- Example:

```
<body>
<p>Display the value of Boolean(10 > 9):</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction()
{
  document.getElementById("demo").innerHTML = Boolean(5 > 3);
}
</script>
</body>
```


Boolean Methods in JavaScript cont...

- Everything with a “value” is TRUE.

```
<body>
<p id="demo"></p>
<script>
var b1 = Boolean(100);
var b2 = Boolean(3.14);
var b3 = Boolean(-15);
var b4 = Boolean("Hello");
var b5 = Boolean('false');
var b6 = Boolean(1 + 7 + 3.14);
document.getElementById("demo").innerHTML =
"100 is " + b1 + "<br>" +
"3.14 is " + b2 + "<br>" +
"-15 is " + b3 + "<br>" +
"Any (not empty) string is " + b4 + "<br>" +
"Even the string 'false' is " + b5 + "<br>" +
"Any expression (except zero) is " + b6;
</script>
</body>
```

Boolean Methods in JavaScript cont...

- Everything without a “value” is FALSE.
- The Boolean value of 0 (zero) is FALSE.
- The Boolean value of -0 (minus zero) is FALSE.
- The Boolean value of “ ”(empty string) is FALSE.
- The Boolean value of undefined is FALSE.
- The Boolean value of null is FALSE.
- The Boolean value of false is FALSE.
- The Boolean value of NaN is FALSE.

Regular Expression in JavaScript

- Regular expressions are patterns used to match character combinations in strings.
- In JavaScript, regular expressions are also objects.
- Regular expressions can be used to perform all types of text search and text replace.
- There are two main string methods: `search()` and `replace()`.
 - `search()` method uses an expression to search for a match and returns the position of the match.
 - `replace()` method uses an expression to return a modified string where the pattern is replaced.

Regular Expression in JavaScript cont...

- Example:

```
<body>
<p id="demo"></p>
<script>
var str = "The quick brown fox jumps over the lazy dog";
var n = str.search(/brown/);
document.getElementById("demo").innerHTML = n;
</script>
</body>
```

```
<body>
<button onclick="myFunction()">Click to replace</button>
<p id="demo">The quick brown fox jumps over the lazy dog</p>
```

Form Method Property

- It is used to change the method for sending form data.
- The method property sets or returns the value of method attribute in a form.
- The syntax for setting the method property is:
formObject.method = get | post
- get – appends the form-data to the URL:
URL?name=value&name=value (this is default).
- post – sends the form-data as an HTTP post transaction.

Form Method Property cont...

- Example:

```
<body>
<h2>Contact Us</h2>
<p>Please fill in this form and send us.</p>
<form action="form2-get.php" method="get">
<label for="inputName">Name:<sup>*</sup></label>
<input type="text" name="name" id="inputName"></br>
<label for="inputEmail">Email:<sup>*</sup></label>
<input type="text" name="email" id="inputEmail"></br>
<label for="inputSubject">Subject:</label>
<input type="text" name="subject" id="inputSubject"></br>
<label for="inputComment">Message:<sup>*</sup></label>
<textarea name="message" id="inputComment" rows="5" cols="30"></textarea></br></br>
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
</body>
```

Form2-get.php:

```
<body>
<h1>ThankYou</h1>
<p>Here is the information you have submitted:</p>
```

JavaScript Arrays

- An array is a special variable that can hold more than one value at a time.
- The Array object is used to store multiple values in a single variable.
- A list of items, storing the fruits in a single variable could be like:

```
var fruit1 = "Apple";  
var fruit2 = "Banana";
```
- This might look possible for few items but isn't feasible for hundreds of item.
- The solution is an array which can replace multiple declaration in a single line like:

```
var fruits = ["Apple", "Banana", "Mango"];
```
- To access an array element, we can refer to the index number like:

```
document.getElementById("demo").innerHTML = fruits[0];
```
- The length property of an array returns the length of an array.
- The push() method is used to add a new element to an array.
- The pop() method is used to remove last element from an array.

JavaScript Arrays (cont...)

- A JavaScript array's length property and numerical properties are connected.
- Several of the built-in array methods (e.g., join(), slice(), indexOf(), etc.) take into account the value of an array's length property when they're called.
- Other methods (e.g., push(), splice(), etc.) also result in updates to an array's length property.
- Example:

```
const fruits = [];  
fruits.push("banana", "apple", "peach");  
console.log(fruits.length); // 3
```


JavaScript Arrays cont...

- Example:

```
<html>
<body>
<p id="demo"></p>
<script>
var fruits = ["Apple", "Banana", "Cherry", "Orange","Mango"];
document.getElementById("demo").innerHTML = "The first element of an array is " + fruits[0];
//document.getElementById("demo").innerHTML = "The last element of an array is " +
fruits[fruits.length-1];
//document.getElementById("demo").innerHTML = "The elements of an array are " + fruits;
//document.getElementById("demo").innerHTML = "The length of an array is " + fruits.length;
/*
fruits.push("Lemon");
document.getElementById("demo").innerHTML = "The elements of an array after adding Lemon
are " + fruits;
*/
</script>
</body>
```

JavaScript Array forEach()

- The forEach() method calls a function for each element in an array.
- The forEach() method is not executed for empty elements.

- Syntax

```
array.forEach(function(currentValue, index, arr), thisValue)
```

- Example:

```
<script>  
let sum = 0;  
const numbers = [65, 44, 12, 4];  
numbers.forEach(myFunction);
```

```
document.getElementById("demo").innerHTML = sum;
```

```
function myFunction(numbers) {  
  sum += numbers;  
}  
</script>
```

JavaScript Array forEach() (cont....)

- Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>JavaScript Arrays</h1>
    <h2>The forEach() Method</h2>
    <p>forEach() calls a function for each element in an array:</p>
    <p>Multiply the value of each element with 10:</p>
    <p id="demo"></p>
    <script>
      const numbers = [65, 44, 12, 4];
      numbers.forEach(myFunction)
      document.getElementById("demo").innerHTML = numbers;
      function myFunction(item, index, arr) {
        arr[index] = item * 10;
      }
    </script>
  </body>
</html>
```

JavaScript Object Constructors

- Sometimes, we need a “blueprint” for creating many objects of same “type”.
- In such situation, the better option to create an “object type” is to use an object constructor function.
- All the objects of same type are created by calling the constructor function with the new keyword.

JavaScript Object Constructors cont...

- Example:

```
<html>
<body>
<h2>JavaScript Object Constructors</h2>
<p id="demo"></p>
<script>
// Constructor function for Person objects
function Person(first, last, age, eye) {
this.firstName = first; this.lastName = last; this.age = age; this.eyeColor = eye;
}
// Create two Person objects
var myFather = new Person("John", "Doe", 50, "blue");
var myMother = new Person("Sally", "Rally", 48, "green");

// Display age
document.getElementById("demo").innerHTML = "My father is " + myFather.age + ". My mother is " + myMother.age + ".";
/*
document.getElementById("demo").innerHTML = "My father name is " + myFather.firstName + " " +
myFather.lastName + ". He is " + myFather.age + " years old with " + myFather.eyeColor + " eyes. My mother
name is " + myMother.firstName + " " + myMother.lastName + ". She is " + myMother.age + " years old with " +
myMother.eyeColor + " eyes.";
*/
</script>
</body>
</html>
```

JavaScript Error Handling

- When executing JavaScript code, different errors can occur.
- Errors can be coding errors made by the programmer, errors due to wrong input, and other unforeseeable things.
- There are four statements to handle errors:
 - The try statement lets you test a block of code for errors.
 - The catch statement lets you handle the error.
 - The throw statement lets you create custom errors.
 - The finally statement lets you execute code, after try and catch, regardless of the result.

JavaScript Error Handling cont...

- The try statement allows you to define a block of code to be tested for errors while it is being executed.
- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.
- The throw statement allows you to create a custom error.
- If you use throw together with try and catch, you can control program flow and generate custom error messages.

- The JavaScript statements try and catch come in pairs:

```
try{  
  //block of code to try  
}  
catch(err) {  
  //block of code to handle errors  
}
```

- Example:

```
<body>  
<p id="demo"></p>  
<script>  
try { alert("Welcome guest!"); }  
catch(err) { document.getElementById("demo").innerHTML = err.message; }  
</script>  
</body>
```

JavaScript Error Handling cont...

- Example:

```
<body>
<p>Please input a number between 5 and 10:</p>
<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>
<p id="p01"></p>
<script>
function myFunction() {
  var message, x;
  message = document.getElementById("p01");
  message.innerHTML = "";
  x = document.getElementById("demo").value;
  try {
    if(x == "") throw "is empty";
    if(isNaN(x)) throw "is not a number";
    x = Number(x);
    if(x > 10) throw "is too high";
    if(x < 5) throw "is too low";
  }
  catch(err) { message.innerHTML = "Input " + err; }
  finally { document.getElementById("demo").value = ""; }
}
</script>
</body>
```


JavaScript Error Handling cont...

- The finally statement lets you execute code, after try and catch, regardless of the result:
- Syntax:
Try
{ //block of code to try }
catch(err)
{ //block of code to handle errors }
Finally
{ //block of code to be executed regardless of the try/catch result }

Form Validation

- It is the process of checking that a form has been filled in correctly before it is processed or not.
- For example, if your form has a box for the user to type their email address, you might want your form handler to check that they've filled in their address before you deal with the rest of the form.
- There are two main methods for validating forms: server-side (using CGI scripts, ASP, etc.), and client-side (usually done using JavaScript).
- Server-side validation is more secure but often trickier to code and it also increases load of server computer, whereas client-side validation is easier to do and quicker too because the browser doesn't have to connect to the server to validate the form, so the user finds out instantly if they've missed out that required field.
- Client-side validation also decreases the load of server computer and hence server computer can focus on business logic processing.

Form Validation: Checking Non-Empty

- Example:

```
<html>
<head>
<script>
function validateForm() {
  var x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Form Validation: Checking Numbers

- Example:

```
<script type='text/javascript'>
function validate()
{
var patt=/^[0-9]+$/;
var v= document.getElementById('elem').value;
if(v.match(patt))
{ alert("valid entry"); }
else
{
alert("Invalid entry:");
document.getElementById('elem').value="";
document.getElementById('elem').focus();
}
}
</script>
<form>
Required Field: <input type='text' id='elem' />
<input type='button' onclick="validate()" value='Check' />
</form>
```

Form Validation: Checking All Letters

- Example:

```
<script type='text/javascript'>
function validate()
{
var patt=/^[a-zA-Z]+$/;
var v= document.getElementById('elem').value;
if(v.match(patt))
{ alert("valid entry"); }
else
{
alert("Invalid entry:");
document.getElementById('elem').value="";
document.getElementById('elem').focus();
}
}
</script>
<form>
Required Field: <input type='text' id='elem' />
<input type='button' onclick="validate()" value='Check' />
</form>
```

Form Validation: Radio Buttons

- Example:

```
<script type='text/javascript'>
function validate()
{
var sex=document.getElementsByName("gen");
if(sex[0].checked==false && sex[1].checked==false)
{ alert("You must choose Gender"); }
else {
if(sex[0].checked==true)
alert("Male");
else
alert("Female");
}
}
</script>
<form>
Select Gender:
<input type=radio name='gen'>Male
<input type=radio name='gen'>Female
<input type='button' onclick='validate()' value='Check'/>
</form>
```

Form Validation: Selection Made

- Example:

```
<script type='text/javascript'>
function validate()
{
var si=document.getElementById('con').selectedIndex;
var v= document.getElementById('con').options[si].text;
if(v=="Please Choose")
{ alert("You must choose the country"); }
else
{ alert("Your Country is:"+v); }
}
</script>
<form>
Select Country: <select id='con'>
<option>Please Choose</option> <option>Nepal</option>
<option>India</option> <option>China</option>
</select>
<input type='button' onclick='validate()' value='Check'/>
</form>
```

Form Validation: Email

- Every email is made up of 5 parts:
 - A combination of letters, numbers, periods, hyphens, plus signs, and/or underscores.
 - The @ symbol.
 - A combination of letters, numbers, hyphens, and/or periods.
 - A period (.).
 - The top level domain (com, net, org, gov, ...).

- Example:

```
<script type='text/javascript'>
function validate()
{
var patt=/^[w\-\.\+]+\@[a-zA-Z0-9\.\-]+\.[a-zA-z0-9]{2,4}$/;
var v= document.getElementById('elem').value;
if(v.match(patt))
{ alert("valid Email"); }
else
{
alert("Invalid Email"); document.getElementById('elem').value="";
document.getElementById('elem').focus();
}
```


Handling Cookies in JavaScript

- Cookies are data, stored in small text files, on your computer.
- When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user.
- Cookies were invented to solve the problem “how to remember information about the user?”
 - When a user visits a web page, his/her name can be stored in a cookie.
 - Next time the user visits the page, the cookie remembers his/her name.
 - Cookies are saved in name-value pairs like: username = John Doe
- When a browser requests a web page from a server, cookies belonging to the page are added to the request; this way the server gets the necessary data to remember information about users.
- JavaScript can create, read, and delete cookies with the **document.cookie** property.

Create a Cookie with JavaScript

- With JavaScript, a cookie can be created like this:
`document.cookie = "username=John Doe";`
- You can also add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:
`document.cookie = "username=John Doe; expires=Thu, 18 Dec 2022 12:00:00 UTC";`
- With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.
`document.cookie = "username=John Doe; expires=Thu, 18 Dec 2022 12:00:00 UTC; path=/";`

Read/Change/Delete a Cookie with JavaScript

- Read a Cookie with JavaScript:

- With JavaScript, a cookie can be read like this:

```
var x = document.cookie;
```

- Change a Cookie with JavaScript:

- With JavaScript, you can change a cookie the same way as you create it:

```
document.cookie = "username=John Smith; expires=Thu, 18 Dec 2022  
12:00:00 UTC; path=/";
```

- The old cookie is overwritten.

- Delete a Cookie with JavaScript:

- Deleting a cookie is very simple as you don't have to specify a cookie value when you delete it.
 - Just set the expires parameter to a passed date:

```
document.cookie = "username; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

Set and get a Cookie with JavaScript

- Example:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type = "button" value = "setCookie" onclick = "setCookie()">
<input type = "button" value = "getCookie" onclick = "getCookie()">
  <script>
    function setCookie()
    {
      document.cookie = "username=Duke Martin";
    }
    function getCookie()
    {
      if(document.cookie.length!=0)
```

Read/Change/Delete a Cookie with JavaScript

- Example:

```
<html>
<head>
  <title>Cookie!!!</title>
  <script type="text/javascript">
    function createCookie(cookieName,cookieValue,daysToExpire){
      var date = new Date();
      date.setTime(date.getTime()+(daysToExpire*24*60*60*1000));
      document.cookie = cookieName + "=" + cookieValue + "; expires=" + date.toGMTString();
    }
    function accessCookie(cookieName){
      var name = cookieName + "=";
      var allCookieArray = document.cookie.split(';');
      for(var i = 0; i<allCookieArray.length; i++)
      {
        var temp = allCookieArray[i].trim();
        if (temp.indexOf(name) == 0)
          return temp.substring(name.length,temp.length);
      }
      return "";
    }
    function checkCookie()
    {
      var user = accessCookie("testCookie");
      if (user == "")
        document.write("No cookie found");
      else
        document.write("Cookie found: " + user);
    }
  </script>
</head>
</html>
```

Document Object Model (DOM)

- The Document Object Model (DOM) is the data representation of the objects that comprise the structure and content of a document on the web.
- It is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content.
- The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.
- A web page is a document that can be either displayed in the browser window or as the HTML source. In both cases, it is the same document but the DOM representation allows it to be manipulated.
- As an object-oriented representation of the web page, it can be modified with a scripting language such as JavaScript.

Why is DOM required?

The Document Object Model (DOM) is essential in web development for several reasons:

- **Dynamic Web Pages:** It allows you to create dynamic web pages. It enables the JavaScript to access and manipulate page content, structure, and style dynamically which gives interactive and responsive web experiences, such as updating content without reloading the entire page or responding to user actions instantly.
- **Interactivity:** With the DOM, you can respond to user actions (like clicks, inputs, or scrolls) and modify the web page accordingly.
- **Content Updates:** When you want to update the content without refreshing the entire page, the DOM enables targeted changes making the web applications more efficient and user-friendly.
- **Cross-Browser Compatibility:** Different browsers may render HTML and CSS in different ways. The DOM provides a standardized way to interact with page elements.
- **Single-Page Applications (SPAs):** Applications built with frameworks such as React or Angular, heavily rely on the DOM for efficient rendering and updating of content within a single HTML page without reloading the full page.

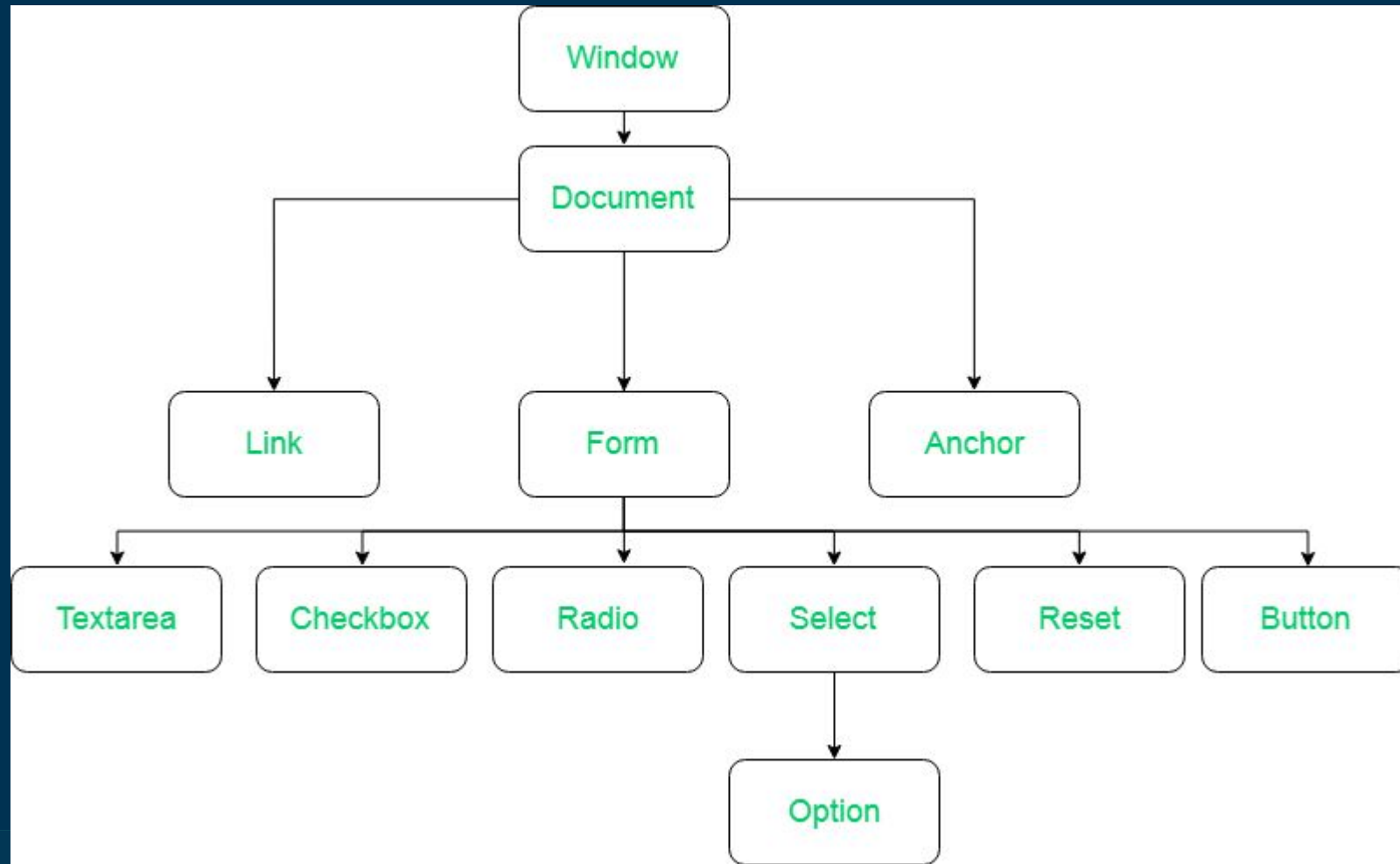
Structure of DOM

- DOM can be thought of as a Tree or Forest(more than one tree).
- The term structure model is sometimes used to describe the tree-like representation of a document. Each branch of the tree ends in a node, and each node contains objects.
- Event listeners can be added to nodes and triggered on an occurrence of a given event.
- Why is DOM called an Object Model?
 - Documents are modeled using objects, and the model includes not only the structure of a document but also the behavior of a document and the objects of which it is composed like tag elements with attributes in HTML.

Properties of DOM

- The properties of the document object that can be accessed and modified by the document object.
 - Window Object: Window Object is object of the browser which is always at top of the hierarchy. It is like an API that is used to set and access all the properties and methods of the browser. It is automatically created by the browser.
 - Document object: When an HTML document is loaded into a window, it becomes a document object. The 'document' object has various properties that refer to other objects which allow access to and modification of the content of the web page. If there is a need to access any element in an HTML page, we always start with accessing the 'document' object. Document object is property of window object.
 - Form Object: It is represented by form tags.
 - Link Object: It is represented by link tags.
 - Anchor Object: It is represented by a href tags.
 - Form Control Elements:: Form can have many control elements such as text fields, buttons, radio buttons, checkboxes, etc.

Properties of DOM



Methods of Document Object

- write("string"): Writes the given string on the document.
- getElementById(): returns the element having the given id value.
- getElementsByName(): returns all the elements having the given name value.
- getElementsByTagName(): returns all the elements having the given tag name.

```
<p>A computer science portal for geeks.</p>
```

```
<button onclick="geek()">Try it</button>
```

```
<script>
```

```
function geek() {  
  var doc = document.getElementsByTagName("p");  
  doc[0].style.background = "green";  
  doc[0].style.color = "white";  
}
```

```
</script>
```

- getElementsByClassName(): returns all the elements having the given class name.

```
<ul class="example">
```

```
<li class="child">Coffee</li>
```

```
<li class="child">Tea</li>
```

```
</ul>
```

Core interfaces in the DOM

- Some of the most commonly-used interfaces in the DOM.
- The following is a brief list of common APIs in web and XML page scripting using the DOM.
 - `document.querySelector()`
 - `document.querySelectorAll()`
 - `document.createElement()`
 - `Element.innerHTML`
 - `Element.setAttribute()`
 - `Element.getAttribute()`
 - `EventTarget.addEventListener()`
 - `HTMLElement.style`
 - `Node.appendChild()`
 - `window.onload`
 - `window.scrollTo()`

Event Handling

- HTML events are "things" that happen to HTML elements.
- When JavaScript is used in HTML pages, JavaScript can "react" on these events.
- These events could be a user scrolling through the page, clicking on an item, or loading a page.
- Here are some common events –
Onclick, dblclick, mousedown, mouseup, mousemove, keydown, keyup, touchmove, touchstart, touchend, onload, onfocus, onblur, onerror, onscroll
- There are two ways to listen to an event:
 - `addEventListener`
 - inline events, such as `onclick`

Event Handling (cont..)

- `//addEventListener`
`document.getElementsByTagName('a').addEventListener('click', onClickHandler);`
- `//inline using onclick`
`<a href="#" onclick="onClickHandler">Click me</a>`
- Which is better - an inline event or `addEventListener`?
 - `addEventListener` gives you the ability to register unlimited event handlers.
 - `removeEventListener` can also be used to remove event handlers
 - The `useCapture` flag can be used to indicate whether an event needs to be handled in the capture phase or bundled phase.

More readings:

<https://www.javascripttutorial.net/javascript-dom/handling-events-in-javascript/>

jQuery

- jQuery is a powerful and widely used JavaScript library to simplify common web scripting task.
- It is based on the principle “write less, do more”.
- The purpose of jQuery is to make it much easier to use JavaScript.
- The jQuery library contains the following features:
 - HTML manipulation
 - CSS manipulation
 - HTML event methods
 - Effects and Animations
 - AJAX
 - Utilities

What can jQuery do?

- Easily select elements to perform manipulations.
- Easily create effect like show or hide elements, sliding transition, and so on.
- Easily create complex CSS animation with fewer lines of code.
- Easily manipulate DOM elements and their attributes.
- Easily implement Ajax to enable asynchronous data exchange between client and server.
- Easily traverse all around the DOM tree to locate any element.
- Easily perform multiple actions on an element with a single line of code.
- Easily get or set dimensions of the HTML elements.
- Advantages of jQuery:
 - Save lots of time
 - Simplify common JavaScript tasks
 - Easy to use
 - Compatible with browsers
 - Absolutely free

jQuery Syntax

- A jQuery statement starts with the dollar sign (\$) and ends with a semicolon (;).
- The dollar sign (\$) is just an alias for jQuery.
- Basic syntax for jQuery is: \$(selector).action();

```
<script>  
$(document).ready(function(){  
  // Some code to be executed...  
  alert("HelloWorld!"); });  
</script>
```

- <script> element – since jQuery is just a JavaScript library, the jQuery code can be placed inside the <script> element. However, if you want to place it in an external JavaScript file, which is preferred, you just remove this part.
- \$(document).ready(handler); – this statement is known as ready event where the handler is basically a function that is passed to the ready() method to be executed safely as soon as the document is ready to be manipulated.
 - This is to prevent any jQuery code from running before the document is finished loading (is ready).
 - It is good practice to wait for the document to be fully loaded and ready before working with it. This also allows you to have your JavaScript code before the body of your document, in the head section.
 - Here are some examples of actions that can fail if methods are run before the document is fully loaded:
 - Trying to hide an element that is not created yet
 - Trying to get the size of an image that is not loaded yet

jQuery Selectors

- jQuery selectors allow us to select and manipulate HTML elements.
- Current Element Selector:
\$(this).hide() – hides the current element.
- Element Selector:
\$("p").hide() – hides all <p> elements.
- .class Selector:
\$(".test").hide() – hides the elements with class = "test".
- #id Selector:
\$("#test").hide() – hides the element with id = "test".
- jQuery Functions In a Separate File:
 - If your website contains a lot of pages, and you want your jQuery functions to be easy to maintain, you can put your jQuery functions in a separate .js file.
 - Example:

```
<head>  
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>  
<script src="my_jquery_functions.js"></script>  
</head>
```

jQuery cont...

- Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $(this).hide();
    //$("p").hide();
    //$("#test").hide();
    //$(".test").hide();
  });
});
</script>
</head>
<body>
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
<p id="test">This is another paragraph with id.</p>
<p class="test">This is another paragraph with class.</p>
<button>Click me to hide paragraphs</button>
</body>
</html>
```

jQuery Event

- All the different visitors' actions that a web page can respond to are called events.
- An event represents the precise moment when something happens.
- Examples:
 - moving a mouse over an element
 - selecting a radio button
 - clicking on an element
- Here are some common DOM events:

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

jQuery Event cont...

- jQuery Syntax For Event Methods:

- In jQuery, most DOM events have an equivalent jQuery method.
- To assign a click event to all paragraphs on a page, you can do this:
`$("p").click();`
- The next step is to define what should happen when the event fires. You must pass a function to the event:
`$("p").click(function(){
 // action goes here!!
});`

- Commonly Used jQuery Event Methods:

- `click()`:
 - The `click()` method attaches an event handler function to an HTML element.
 - The function is executed when the user clicks on the HTML element.
 - The following example says: When a click event fires on a `<p>` element; hide the current `<p>` element:
`$("p").click(function(){
 $(this).hide();
});`

jQuery Event cont...

- `dblclick()`:

- The `dblclick()` method attaches an event handler function to an HTML element.
- The function is executed when the user double-clicks on the HTML element:

```
$("#p").dblclick(function(){  
    $(this).hide();  
});
```

- `mouseenter()`:

- The `mouseenter()` method attaches an event handler function to an HTML element.
- The function is executed when the mouse pointer enters the HTML element:

```
$("#p1").mouseenter(function(){  
    alert("You entered p1!");  
});
```

jQuery Event cont...

- `mouseleave()`:

- The `mouseleave()` method attaches an event handler function to an HTML element.
- The function is executed when the mouse pointer leaves the HTML element:

```
$("#p1").mouseleave(function(){  
    alert("Bye! You now leave p1!");  
});
```

- `hover()`:

- The `hover()` method takes two functions and is a combination of the `mouseenter()` and `mouseleave()` methods.
- The first function is executed when the mouse enters the HTML element, and the second function is executed when the mouse leaves the HTML element:

```
$("#p1").hover(function(){  
    alert("You entered p1!");  
},  
function(){  
    alert("Bye! You now leave p1!");  
});
```

jQuery Event cont...

- **focus():**

- The focus() method attaches an event handler function to an HTML form field.
- The function is executed when the form field gets focus:

```
$("#input").focus(function(){  
    $(this).css("background-color", "#cccccc");  
});
```

- **blur():**

- The blur() method attaches an event handler function to an HTML form field.
- The function is executed when the form field loses focus:

```
$("#input").blur(function(){  
    $(this).css("background-color", "#ffffff");  
});
```


jQuery Event cont...

- jQuery Mouse Events Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<style>
p{ padding:20px; font: 20px sans-serif; background:aqua; }
</style>
<script>
$(document).ready(function(){
  $("p").click(function(){
    $(this).slideUp();
  });
});
</script>
</head>
<body>
<p>Click on me and I'll disappear.</p>
<p>Click on me and I'll disappear.</p>
<p>Click on me and I'll disappear.</p>
</body>
</html>
```

jQuery Event cont...

- jQuery Keyboard Events Example

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<style>
p{ padding: 10px; background: lightgreen; display: none; }
div{ margin: 20px 0; }
</style>
<script>
$(document).ready(function(){
  var i = 0;
  $('input[type="text"]').keypress(function(){
    $("span").text(i += 1);
    $("p").show().fadeOut();
  });
});
</script>
```

jQuery Event cont...

- jQuery Form Events Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<script>
$(document).ready(function(){
  $("select").change(function(){
    var selectedOption = $(this).find(":selected").val();
    alert("You have selected - " + selectedOption);
  });
});
</script>
</head>
<body>
<form>
<label>City:</label>
<select>
<option>Kathmandu</option>
<option>Pokhara</option>
<option>Chitwan</option>
</select>
</form>
<p><strong>Note:</strong> Select any value from the dropdown select and see the result.</p>
</body>
</html>
```

jQuery Event cont...

- jQuery Form Events Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<style>
label{ display: block; margin: 5px 0; }
label span{ display: none; }
</style>
<script>
$(document).ready(function(){
  $("input").focus(function(){
    $(this).next("span").show().fadeOut("slow");
  });
});
</script>
</head>
<body>
<form>
<label>Email: <input type="text"> <span>focus fire</span></label>
<label>Password: <input type="password"> <span>focus fire</span></label>
<label><input type="submit" value="Sign In"> <span>focus fire</span></label>
</form>
<p><strong>Note:</strong> Click on the form control or press the "Tab" key to set focus.</p>
</body>
</html>
```

jQuery Event cont...

- jQuery Window Events Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<style>
p{ padding: 20px;font: 20px sans-serif;background:#f0e68c;}
</style>
<script>
$(document).ready(function(){
  $(window).resize(function() {
    $(window).bind("resize",function(){
      $("p").text("Window width: " + $(window).width() + ", " + "Window height: " + $(window).height());
    });
  });
});
</script>
</head>
<body>
<p>Open the output in a new tab and resize the browser window by dragging the corners.</p>
</body>
</html>
```

jQuery Event cont...

- jQuery Window Events Example:

```
<html>
<head>
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
<style>
p{ width: 100%; padding: 50px 0; text-align: center; font: bold 34px sans-serif;
background:
#f0e68c; position: fixed; top: 50px; display: none; }
.dummy-content{ height: 600px; font: 34px sans-serif; text-align: center; }
</style>
<script>
$(document).ready(function(){
$(window).scroll(function() {
$("p").show().fadeOut("slow");
});
});
</script>
</head>
<body>
```

The on() Method

- The on() method attaches one or more event handlers for the selected elements.
- Attach a click event to a <p> element:

```
$( "p" ).on( "click", function() {  
    $( this ).hide();  
});
```
- Attach multiple event handlers to a <p> element:

```
$( "p" ).on( {  
    mouseenter: function() {  
        $( this ).css( "background-color", "lightgray" );  
    },  
    mouseleave: function() {  
        $( this ).css( "background-color", "lightblue" );  
    },  
    click: function() {  
        $( this ).css( "background-color", "yellow" );  
    }  
});
```

jQuery Effects

- jQuery hide() and show():
 - With jQuery, you can hide and show HTML elements with the hide() and show() methods:

```
$("#hide").click(function(){
    $("p").hide();
});
$("#show").click(function(){
    $("p").show();
});
```

- Example:

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("#hide").click(function(){
        $("p").hide();
    });
});
```


jQuery Effects cont...

Method	Description
<u>animate()</u>	Runs a custom animation on the selected elements
<u>clearQueue()</u>	Removes all remaining queued functions from the selected elements
<u>delay()</u>	Sets a delay for all queued functions on the selected elements
<u>dequeue()</u>	Removes the next function from the queue, and then executes the function
<u>fadeIn()</u>	Fades in the selected elements
<u>fadeOut()</u>	Fades out the selected elements
<u>fadeTo()</u>	Fades in/out the selected elements to a given opacity
<u>fadeToggle()</u>	Toggles between the fadeIn() and fadeOut() methods
<u>finish()</u>	Stops, removes and completes all queued animations for the selected elements
<u>hide()</u>	Hides the selected elements
<u>queue()</u>	Shows the queued functions on the selected elements
<u>show()</u>	Shows the selected elements
<u>slideDown()</u>	Slides-down (shows) the selected elements
<u>slideToggle()</u>	Toggles between the slideUp() and slideDown() methods
<u>slideUp()</u>	Slides-up (hides) the selected elements
<u>stop()</u>	Stops the currently running animation for the selected elements
<u>toggle()</u>	Toggles between the hide() and show() methods

Introduction to JSON(JavaScript Object Notation)

The JSON format was originally specified by Douglas Crockford.

- JSON stands for JavaScript Object Notation
- The JSON files are lightweight and take less storage to be stored.
- These files can be usually created and edited by a text editor. These text editors are mostly freely available.
- JSON files are human-readable means the user can read them easily.
- JSON is used to send data between computers
- Almost every programming language supports JSON format because they have libraries and functions to read/write JSON structures.
- Why Use JSON?
 - The JSON format is syntactically similar to the code for creating JavaScript objects. Because of this, a JavaScript program can easily convert JSON data into JavaScript objects.
 - Since the format is text only, JSON data can easily be sent between computers, and used by any programming language.
 - JavaScript has a built in function for converting JSON strings into JavaScript objects:
JSON.parse()
 - JavaScript also has a built in function for converting an object into a JSON string:
JSON.stringify()

JSON (JavaScript Object Notation)

- Syntax Rules
 - Data is in name/value pairs
 - Data is separated by commas
 - Curly braces hold objects
 - Square brackets hold arrays
- JSON Data:
 - JSON data is written as name/value pairs, just like JavaScript object properties.
 - A name/value pair consists of a field name (in double quotes), followed by a colon, followed by a value:
`"firstName":"John"`
- JSON Objects:
 - JSON objects are written inside curly braces.
 - Just like in JavaScript, objects can contain multiple name/value pairs:
 - `{"firstName":"John", "lastName":"Doe"}`

JSON cont...

- JSON Arrays:

- JSON arrays are written inside square brackets.
- Just like in JavaScript, an array can contain objects:

```
"employees":[  
  {"firstName":"John", "lastName":"Doe"},  
  {"firstName":"Anna", "lastName":"Smith"},  
  {"firstName":"Peter", "lastName":"Jones"}  
]
```

- In the example above, the object "employees" is an array. It contains three objects.
- Each object is a record of a person (with a first name and a last name).

JSON vs. XML

JSON

JSON object has a type

JSON types: string, number, array, Boolean

Data is readily accessible as JSON objects

JSON files are more human-readable.

JSON is supported by most browsers.

JSON has no display capabilities.

Retrieving value is easy

Supported by many Ajax toolkit

A fully automated way of deserializing/serializing JavaScript.

Native support for object.

XML

XML data is typeless

All XML data should be string

XML data needs to be parsed.

XML files are less human-readable.

Cross-browser XML parsing can be tricky

XML provides a capability to display data because it is a markup language.

Retrieving value is difficult

Not fully supported by Ajax toolkit

Developers have to write JavaScript code to serialize/de-serialize from XML

The object has to be express by conventions - mostly missed use of attributes and elements.

JSON cont...

- Converting a JSON Text to a JavaScript Object:

- A common use of JSON is to read data from a web server and display the data in a web page.
- For simplicity, this can be demonstrated using a string as input.
- First, create a JavaScript string containing JSON syntax:

```
var text = '{ "employees" : [' +  
  '{ "firstName":"John" , "lastName":"Doe" },' +  
  '{ "firstName":"Anna" , "lastName":"Smith" },' +  
  '{ "firstName":"Peter" , "lastName":"Jones" } ]}';
```

- Then, use the JavaScript built-in function `JSON.parse()` to convert the string into a JavaScript object:

```
var obj = JSON.parse(text);
```

- Finally, use the new JavaScript object in your page:

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML =
```

```
obj.employees[1].firstName + " " + obj.employees[1].lastName;
```

```
</script>
```

JSON cont...

- Example:

```
<!DOCTYPE html>
<html>
<body>
  <h2>Create Object from JSON String</h2>
  <p id="demo"></p>
  <script>
    var text = '{"employees":[' +
    '{"firstName":"John","lastName":"Doe" },' +
    '{"firstName":"Anna","lastName":"Smith" },' +
    '{"firstName":"Peter","lastName":"Jones" }]}';

    obj = JSON.parse(text);
    document.getElementById("demo").innerHTML =
    obj.employees[1].firstName + " " + obj.employees[1].lastName;
  </script>

</body>
</html>
```

JSON cont...

- Example:

```
<?php
```

```
    // Read the JSON file
```

```
    $json = file_get_contents('my_data.json');
```

```
    // Decode the JSON file
```

```
    $json_data = json_decode($json,true);
```

```
    // Display data
```

```
    print_r($json_data);
```

```
?>
```


Introduction to AJAX

- Asynchronous JavaScript And XML
- Not a stand-alone language or technology
- Combines a set of known technologies in order to create faster and more user-friendly web pages
- Client-side technology
- Makes web pages more responsive by exchanging small amounts of data
- AJAX allows web pages to be updated asynchronously by exchanging data with a web server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.
- Web browser technology independent of web server software
- AJAX just uses a combination of:
 - A browser built-in XMLHttpRequest object (to request data from a web server)
 - JavaScript and HTML DOM (to display or use the data)

Introduction to AJAX cont...

- It is a term coined in 2005 by Jesse James Garrett, that describes a "new" approach to using a number of existing technologies together, including HTML or XHTML, CSS, JavaScript, DOM, XML, XSLT, and most importantly the XMLHttpRequest object.
- When these technologies are combined in the Ajax model, web applications can make quick, incremental updates to the user interface without reloading the entire browser page. This makes the application faster and more responsive to user actions.
- Although X in Ajax stands for XML, JSON is used more than XML nowadays because of its many advantages such as being lighter and a part of JavaScript. Both JSON and XML are used for packaging information in the Ajax model.

AJAX Example

- HTML

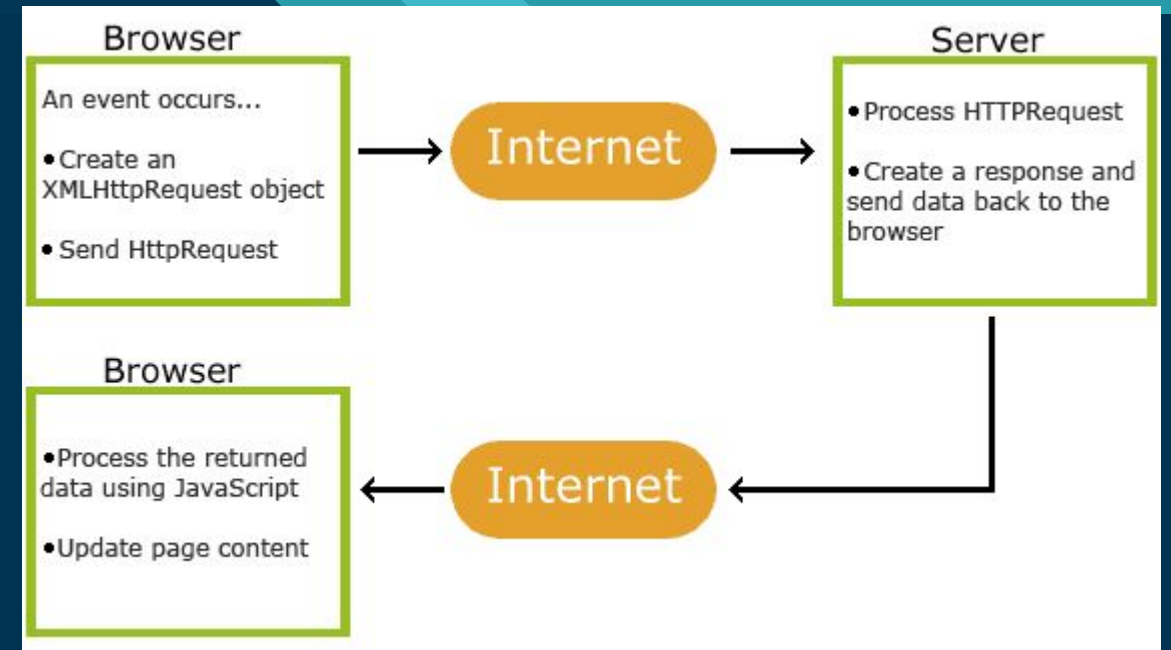
```
<!DOCTYPE html>
<html>
<body>
<div id="demo">
  <h2>Let AJAX change this text</h2>
  <button type="button" onclick="loadDoc()">Change Content</button>
</div>
</body>
</html>
```

- Function loadDoc()

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhttp.open("GET", "ajax info.txt", true);
```

How AJAX Works

1. An event occurs in a web page (the page is loaded; a button is clicked)
2. An XMLHttpRequest object is created by JavaScript
3. The XMLHttpRequest object sends a request to a web server
4. The server processes the request
5. The server sends a response back to the web page
6. The response is read by JavaScript
7. Proper action (like page update) is performed by JavaScript



Ajax cont...

- **The XMLHttpRequest Object**

- The XMLHttpRequest object can be used to exchange data with a server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.

- Syntax for creating an XMLHttpRequest object:

```
variable = new XMLHttpRequest();
```

- new XMLHttpRequest(), creates a new XMLHttpRequest object.

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the status-number of a request 200: "OK" 403: "Forbidden" 404: "Not Found" For a complete list go to the Http Messages Reference
statusText	Returns the status-text (e.g. "OK" or "Not Found")

Problems with Conventional Web Applications

- Interruption of user operation
 - Users cannot perform any operation while waiting for a response.
- Loss of operational context during refresh
 - Loss of information on the screen
 - Loss of scrolled position
- No instant feedbacks to user activities
 - A user must wait for the next page
- Constrained by HTML
 - Lack of useful widgets

Advantages of AJAX

- Page can be refreshed dynamically
- Response of the interface to user is faster
- Load much faster because the payload is much smaller
- Reduces the demand for bandwidth
- Allows the web applications to be much more efficient
- Operate more like an application rather than a standalone program

Conclusion for AJAX

- AJAX provides functionality to create a robust web application.
- If an AJAX web application is coded properly it will run faster than and as secure as a non-AJAX program.
- AJAX also allows websites to reduce their overall bandwidth usage and server load by reducing the amount of full-page loads.
- Several applications using AJAX includes the following:
 - Google Map
 - Gmail
 - YouTube
 - Facebook